DAA LAB PRACTICALS

Explanation of all five Python programs

Subject: Design and Analysis of Algorithms

This guide explains the uploaded programs in simple, practical-oriented language. For each program, it covers the purpose, working, important statements, sample result, complexity and useful viva points.

Prepared from the five Python programs supplied in this conversation.

1. Fibonacci – Recursive and Non-Recursive

Practical question: Write recursive and non-recursive programs to calculate Fibonacci numbers and analyse their time and space complexity.

What the program does

The Fibonacci sequence starts with 0 and 1. Every next term is the sum of the previous two terms: 0, 1, 1, 2, 3, 5, 8, 13, ... The program prints the requested number of terms using two approaches.

A. Non-recursive (iterative) approach

• **n1, n2 = 0, 1:** Stores the first two Fibonacci terms.

• **count = 0:** Tracks how many terms have been printed.

• **Input validation:** If the number of terms is 0 or negative, the program displays an error message. If it is 1, it prints only 0.

• **while count < nterms:** Repeats until the required number of terms has been printed.

• **print(n1):** Prints the current term.

• **nth = n1 + n2:** Calculates the next term.

• **n1 = n2; n2 = nth:** Moves the two variables forward to the next pair.

• **count += 1:** Increases the number of printed terms by one.

The loop avoids recursion. It only keeps two previous values, so it uses constant auxiliary space.

B. Recursive approach

• **recur_fibo(n):** A function that returns the Fibonacci value at position n.

• **if n <= 1: return n:** These are the base cases. The function directly returns 0 or 1.

• **return recur_fibo(n-1) + recur_fibo(n-2):** For larger n, it calls itself for the two previous positions and adds the results.

• **for i in range(nterms):** Calls the recursive function for positions 0 through nterms – 1 and prints each result.

The base cases stop the recursive calls. Without them, the function would keep calling itself.

Complexity analysis

Approach	Time complexity	Auxiliary space	Reason
Iterative	O(n)	O(1)	One loop; only two values are stored.
Recursive	O(2■)	O(n)	Repeated recursive calls; maximum call depth is n.

Sample: If the user enters 6, both methods print: 0, 1, 1, 2, 3, 5.

Viva points

• A base case is the condition that stops recursion.

• The iterative method is more efficient for this basic Fibonacci calculation.

• The recursive version repeats many calculations, which causes its exponential time complexity.

2. Huffman Encoding using Greedy Strategy

Practical question: Write a program to implement Huffman Encoding using a greedy strategy.

What the program does

Huffman Encoding is a lossless compression technique. It assigns shorter binary codes to symbols that occur more often and longer codes to symbols that occur less often. The program builds a binary tree by repeatedly combining the two symbols or nodes with the smallest frequencies.

Explanation of the code

• **Node class:** Represents a node in the Huffman tree. It stores the frequency (prob), symbol, left child, right child and a code bit.

• **Calculate_Probability(data):** Counts how many times each character appears in the input string and returns a dictionary of character frequencies.

• **Calculate_Codes(node, val):** Traverses the Huffman tree. It adds the current node's bit to the code. Moving left adds 0 and moving right adds 1. When a leaf node is reached, its complete code is stored for that symbol.

• **Output_Encoded(data, coding):** Looks up each character's Huffman code, prints it and joins the codes into one encoded string.

• **Total_Gain(data, coding):** Calculates the original size assuming 8 bits per character, then calculates the compressed size using each character's frequency multiplied by its code length.

• **Huffman_Encoding(data):** This is the main function. It counts frequencies, creates one node per symbol and builds the tree.

Greedy tree-building steps

• Sort the current nodes in ascending order of frequency.

• Select the two nodes with the smallest frequencies.

• Assign 0 to the left node and 1 to the right node.

• Create a new parent node whose frequency is the sum of the two selected frequencies.

• Remove the two selected nodes and insert the new parent node.

• Repeat until only one node remains. This node is the root of the Huffman tree.

Why it is greedy

At every step, the algorithm makes the locally best choice: it combines the two least-frequent available nodes. Repeating this process constructs the Huffman tree.

Complexity and practical note

• **Frequency counting:** O(n), where n is the number of characters.

• **Tree construction in this exact code:** The list is sorted again in every iteration. With k distinct symbols, this takes $O(k^2 \log k)$ time as a simple upper-bound analysis.

• **Encoding:** O(n) for looking up and appending one code per input character, excluding the additional repeated counting performed in Total_Gain.

The code's Total_Gain function calls data.count(symbol) for each distinct symbol, which can take O(nk) time in total. A priority queue can make standard Huffman tree construction more efficient, but this uploaded program uses repeated sorting.

Viva points and limitations

• Huffman coding is lossless: decoding can recover the original data.

• The codes are prefix-free: no symbol's code is the prefix of another symbol's code.

• The uploaded example works. Empty input and input containing only one distinct symbol need special handling in a more general version.

3. Fractional Knapsack using Greedy Method

Practical question: Write a program to solve a Fractional Knapsack problem using a greedy method.

What the program does

The knapsack has a fixed capacity. Each item has a value and a weight. In Fractional Knapsack, an item can be divided, so the program may take only part of an item when the remaining capacity is not enough for the whole item.

Explanation of the code

• **Item class:** Stores the value and weight of each item.

• **fractionalKnapsack(W, arr):** Receives the knapsack capacity W and the list of items.

• **arr.sort(..., reverse=True):** Sorts items in descending order of value-to-weight ratio. The ratio tells how much value an item gives per unit of weight.

• **finalvalue = 0.0:** Stores the total value collected.

• **if item.weight <= W:** If the complete item fits, take it, subtract its weight from W and add its full value.

• **else:** If the complete item does not fit, take the fraction that fits. Its value is item.value × (remaining capacity / item.weight).

• **break:** After taking the fraction, the knapsack is full, so no more items can be added.

• **return finalvalue:** Returns the maximum value obtained by this greedy selection.

Why it is greedy

The program always chooses the available item with the highest value-to-weight ratio first. This is the greedy choice for the Fractional Knapsack problem.

Sample calculation

Item	Value	Weight	Value/weight
1	60	10	6
2	100	20	5
3	120	30	4

For capacity 50, the program takes item 1 (weight 10, value 60), item 2 (weight 20, value 100), and 20/30 of item 3. The value from the fraction is 120 × 20/30 = 80. Total value = 60 + 100 + 80 = 240.

Complexity analysis

• **Time:** O(n log n) for sorting, followed by O(n) to scan the items. Overall: O(n log n).

• **Auxiliary space:** The sorting implementation may use additional space; the algorithm itself uses O(1) extra variables apart from sorting.

Viva points

• Fractional Knapsack allows an item to be divided; 0/1 Knapsack does not.

• The greedy choice is based on value/weight ratio, not simply the highest value.

• The greedy method gives an optimal solution for Fractional Knapsack.

4. 0/1 Knapsack using Dynamic Programming

Practical question: Write a program to solve a 0/1 Knapsack problem using Dynamic Programming or Branch and Bound.

What the program does

Each item has a value and a weight, and the knapsack has capacity W. In 0/1 Knapsack, each item must either be selected completely or not selected at all. Fractions are not allowed.

Meaning of the DP table

K[i][w] represents the maximum value that can be obtained using the first i items with a capacity of w.

Explanation of the code

• **knapSack2D(W, wt, val, n):** Receives capacity, item weights, item values and number of items.

• **K = [[0 ...]]:** Creates a table with n+1 rows and W+1 columns, initially filled with zero. Row 0 represents no items; column 0 represents zero capacity.

• **Nested loops:** Visit every item count i and every capacity w to fill the table.

• **Base condition:** If i == 0 or w == 0, the maximum value is 0.

• **wt[i-1] <= w:** Checks whether the current item fits in the current capacity.

• **max(val[i-1] + K[i-1][w-wt[i-1]], K[i-1][w]):** Compares two choices: include the current item, or exclude it. The maximum of the two is stored.

• **Else:** If the item does not fit, copy the value from the row above: K[i-1][w].

• **res = K[n][W]:** Stores the best value for all items and the full capacity.

• **Backtracking loop:** Moves from the bottom-right of the table towards the first row to find which items were selected. If the value is unchanged from the row above, the item was not selected. Otherwise, the item is selected and its value and weight are subtracted.

• **return K[n][W]:** Returns the maximum obtainable value.

The recurrence relation

If the current item fits:
K[i][w] = max(val[i−1] + K[i−1][w−wt[i−1]], K[i−1][w])
If it does not fit:
K[i][w] = K[i−1][w]

Sample result

The uploaded example has values [3, 4, 5, 6], weights [2, 3, 4, 5] and capacity 5. The best selection is item 1 and item 2 (1-indexed): total weight = 5 and total value = 7.

Complexity analysis

• **Time:** O(nW), because the table has (n+1)(W+1) cells and each cell takes constant time to calculate.

• **Space:** O(nW) for the 2D DP table. The selected-items list and loop use additional space up to O(n).

Viva points

• Dynamic Programming solves smaller subproblems and stores their results in a table.

• The 0/1 restriction means an item cannot be selected more than once.

- The program uses the Dynamic Programming option allowed by the practical question.

5. N-Queens using Backtracking with First Queen Placed

Practical question: Design an n-Queens matrix having the first Queen placed. Use backtracking to place the remaining Queens and generate the final matrix.

What the program does

The N-Queens problem places N queens on an N × N chessboard so that no two queens attack each other. Queens must not share the same row, column or diagonal. In this program, the user places the first queen and the program attempts to place the remaining queens.

Explanation of the code

• **print_board(board, n):** Prints every row and column of the board. A 1 represents a queen and a 0 represents an empty square.

• **is_safe(board, row, col, n):** Checks whether a queen can be placed at the given position.

• **Row check:** Scans the selected row and rejects the position if another queen is present.

• **Column check:** Scans the selected column and rejects the position if another queen is present.

• **Diagonal checks:** Checks upper-left, lower-left, upper-right and lower-right directions. If any queen is found, the position is unsafe.

• **return True:** If none of the checks finds another queen, the position is safe.

• **solve_n_queens_with_first_placed(...):** Recursively tries to place queens column by column.

• **if col >= n:** All columns have been processed, so a solution has been found.

• **if col == first_col:** Skips the column containing the already placed first queen.

• **for row in range(n):** Tries each row in the current column.

• **if is_safe(...):** If the position is safe, places a queen by setting board[row][col] = 1.

• **Recursive call:** Attempts to place a queen in the next column.

• **Backtracking:** If the recursive call fails, removes the queen by resetting the cell to 0 and tries another row.

• **Driver code:** Takes N and the first queen's row and column from the user, places that queen, prints the initial board and calls the solving function.

How backtracking works

The program makes a choice by placing a queen. It then explores the next column. If that choice prevents a complete solution, it undoes the choice and tries another row. This is called backtracking.

Complexity and practical notes

• **Safety check:** O(n), because it scans rows, columns and diagonals.

• **Worst-case search:** The search is factorial in nature; a simple upper bound is $O(n \cdot n!)$ for the safety checks across the search tree.

• **Space:** $O(n^2)$ for the board, plus O(n) recursive call depth.

The program assumes the entered first-queen coordinates are valid and does not validate them before indexing the board. It also finds one solution and stops, rather than printing every possible solution. Some fixed first-queen positions may have no solution.

Viva points

• Backtracking tries a choice and reverses it if it leads to failure.

- A queen attacks horizontally, vertically and diagonally.
- The program places one queen per column and skips the column of the first queen.

Quick Revision for Practical Viva

Program	Technique	Main idea	Time complexity
Fibonacci iterative	Iteration	Add previous two terms	O(n)
Fibonacci recursive	Recursion	F(n)=F(n–1)+F(n–2)	O(2■)
Huffman	Greedy	Combine two least frequencies	This code: O(k² log k) tree build
Fractional Knapsack	Greedy	Choose highest value/weight ratio	O(n log n)
0/1 Knapsack	Dynamic Programming	Compare include vs exclude	O(nW)
N-Queens	Backtracking	Place, test, undo if needed	Factorial search

Remember: In the practical, explain what each function does, describe the main algorithm steps, run the program with a sample input and be ready to explain its time and space complexity.